

Supply Chain Security Roadmap

Status: Draft

Current focus: Developers must only pull packages from internally managed package repositories.

Purpose

Bayview's near-term supply chain security roadmap is focused on one control: stop developers from resolving software packages directly from public package repositories such as npm, PyPI, Maven Central, NuGet, RubyGems, Go module proxies, Cargo, public container registries, and similar external sources.

Today, many developers pull packages directly from the internet. If a trusted public package, child dependency, maintainer account, or registry path is compromised, malicious code may execute in developer environments and expose developer secrets. Reactive defenses such as SCA scanning are still useful, but they do not prevent the initial package resolution path from reaching developer machines.

This roadmap narrows the work to package source control. Broader supply chain topics such as sandboxed development, full incident response, container hardening, credential redesign, and general vulnerability remediation can remain in other AppSec workstreams.

This roadmap expands on [Supply Chain Security](#) and the working notes in [SCRATCH Developer Supply Chain Security](#).

Problem Statement

"Implementing control over software development package management resolution, and preventing data infil/exfil."

Currently, we are exposed to software supply chain risk as our developers pull directly from internet package repositories like PyPI and NPM. If we are hit by a "watering-hole attack" such as Shai-Hulud, we would have our developer secrets compromised or full compromise of the application or endpoint. We currently only have reactive defenses that can come in after-the-compromise such as Wiz SCA scanning.

We can solve this by preventing developers from pulling packages from external package repositories, and only allowing them to pull from internally managed package repositories, which observe controls like dependency cooldowns and package scanning.

Requirement

1. Developers must only pull packages from internally managed package repositories.

For this roadmap, an internally managed package repository may be:

- A hosted internal repository for Bayview-owned packages.
- A proxy, mirror, or grouped repository managed by Bayview for approved public packages.
- An approved cleanroom, hardened, or curated package source connected through Bayview-controlled repository management.
- An approved internal container registry or base image repository.

It does not include direct developer access to public package registries or random vendor download URLs unless there is a documented, time-bound exception.

Scope

In scope

- Identify coding languages, package managers, package ecosystems, and developer workflows in use.
- Select and configure internally managed repositories for package resolution.
- Migrate developer package manager configuration to internal repositories.
- Block, alert on, or otherwise prevent direct external package repository access.
- Define exceptions, ownership, monitoring, and success metrics.
- Update the Secure Software Standard to require internal package repositories.

Out of scope for this focused roadmap

- A complete dependency approval program for every package and version.
- A full sandboxed development mandate.
- A broad local laptop development ban.
- General SCA remediation and CVE management.
- Full supply chain incident response runbooks.
- Enterprise container hardening outside of internal image source control.

Target State

- Developers resolve packages only from approved internal repositories.
- Public package repositories are blocked by default for developer package manager traffic.
- Internal repositories provide package caching, access control, audit logs, and a central place to enforce policy.
- A repository firewall or equivalent package-risk policy layer evaluates packages before they become available to developers through internal repositories.
- Dependency cooldowns or equivalent delay controls prevent newly published public package versions from being consumed immediately.
- Common ecosystems have documented setup instructions and default configuration.
- New or missing package requests have a lightweight intake path.
- Exceptions are time-bound, owned, reviewed, and visible to AppSec and platform teams.
- Repository usage can be measured by team, application, ecosystem, and package source.

Guiding Principles

- **Control package resolution first.** The first milestone is not perfect package judgment; it is making sure packages enter developer workflows through Bayview-managed paths.
- **Make the internal path usable.** Developers need working package manager configuration, documentation, common package availability, and fast support.
- **Block with visibility.** Start with logging and pilot enforcement where needed, but the end state is to prevent direct external repository use.
- **Prefer a consistent platform pattern.** Artifactory and Nexus solve similar repository-management problems; Bayview should avoid duplicate enterprise patterns unless there is a clear reason.

- **Use equivalent capabilities where appropriate.** Sonatype tools are named because they directly support the repository-control pattern, but equivalent tools are acceptable if they meet Bayview's control, logging, enforcement, and developer experience requirements.
- **Keep exceptions narrow.** Exceptions should identify the owner, ecosystem, package source, reason, compensating controls, and expiration date.

Required Capabilities

The roadmap needs these capabilities, whether delivered by the named product or an equivalent tool.

Capability	Example Tool	Why Bayview Needs It
Internal package repository management	Sonatype Nexus Repository or equivalent	Developers need a Bayview-managed place to resolve packages from, so package access can be centralized, cached, logged, authenticated, and disconnected from direct public registry access.
Repository firewall / package admission control	Sonatype Repository Firewall or equivalent	Blocking public registries is necessary but not sufficient; Bayview also needs a control point that can evaluate, quarantine, or block risky packages before they enter the internal repository path.
Dependency cooldowns	Dependency cooldown tooling, repository policy, or equivalent	Newly published package versions are high-risk during the first hours or days after release; cooldowns give maintainers, vendors, and the security community time to detect malicious or compromised releases before Bayview developers consume them.

Supported Package Managers

The initial SCA language inventory should drive which package managers and repository formats Bayview supports first. These counts are a starting signal for prioritization, not a substitute for repository-level discovery of manifests, lockfiles, CI/CD scripts, Dockerfiles, and developer workstation configuration.

SCA Language	Count	Package Managers and Sources Commonly Found With This Language
Unknown	47584	Anomaly. These are mostly DPKG/OS level packages, not actual lockfiles.
PYTHON	1795	pip , pip-tools , Poetry , Pipenv , PDM , uv , Conda , and private Python package indexes.
GO	1625	Anomaly. These are all OS packages. We do not develop in Go.
CSHARP	1462	NuGet , dotnet restore , Visual Studio package restore, private NuGet feeds, and Azure Artifacts feeds where applicable.
JAVASCRIPT	1245	npm , Yarn , pnpm , private npm registries, and package manager lockfiles such as package-lock.json , yarn.lock , and pnpm-lock.yaml .

SCA Language	Count	Package Managers and Sources Commonly Found With This Language
JAVA	1017	Maven, Gradle, private Maven repositories, internal artifact repositories, and repository settings in <code>settings.xml</code> , <code>pom.xml</code> , and Gradle configuration.
RUBY	32	Anomaly. These are all container packages, not Ruby package manager lockfiles.
RUST	1	Anomaly. This is an OS package, not Rust package manager lockfiles.

Cross-cutting package sources such as public container registries, operating system package managers, and curl-to-shell installers should be handled even when they do not map cleanly to a single SCA language.

Stakeholder Alignment

Initial stakeholder groups and owners from the working notes include Nick Akl's organization, Steve Dixon, Wing Chau, Matt Miller, Wei Zhang, Keith Nam, Marcelo Olivas, Henry Post, Jay Rosario, Chet Heacox, Dan F, Jared Stoll, Cheryl Klein, and Jay Pearlman.

The stakeholder group should confirm:

- Which team owns the enterprise repository platform.
- Which team owns developer endpoint or network enforcement.
- Which team owns package-source exceptions.
- Which engineering leaders will sponsor developer migration.
- Where the Secure Software Standard update will be tracked.

Tool Roles

These tools are complementary, not interchangeable. The focused roadmap should prioritize a repository manager first; other tools may be useful as upstream sources or policy layers.

Tool or Option	Role in This Roadmap	Best Fit	What It Does Not Replace
JFrog Artifactory	Enterprise repository manager for hosted, proxy, cached, and grouped package repositories across many ecosystems.	Primary internal package source if Bayview wants one broad artifact and package platform across npm, PyPI, Maven, NuGet, containers, and other formats.	SCA, source code review, developer endpoint controls, or incident response.
Sonatype Nexus Repository or equivalent	Enterprise repository manager for hosted, proxy, cached, and grouped package repositories across common ecosystems.	Primary internal package source if Bayview prefers Sonatype workflows or has Maven-heavy practices. The reason to include it is to give developers a controlled package resolution path that can be logged, governed, and blocked from direct public registry access.	SCA, source code review, developer endpoint controls, or incident response.

Tool or Option	Role in This Roadmap	Best Fit	What It Does Not Replace
Cloud-native package registries	Internal package hosting tied to a cloud or DevOps platform.	Specific ecosystems or teams where the cloud-native registry is already the standard and can meet Bayview policy.	A universal enterprise repository strategy across all ecosystems.
Sonatype Repository Firewall or equivalent	Repository firewall and package admission-control layer that can evaluate packages before or as they enter the internal repository path.	Useful where Bayview wants to quarantine or block malware, suspicious components, dependency confusion risk, or policy-violating packages before developers can install them from internal repositories.	The repository manager itself, endpoint egress blocking, or developer package manager configuration.
Dependency cooldowns or equivalent	Delay newly published package versions before they are available through internal repositories.	Useful for npm, PyPI, and other fast-moving ecosystems where malicious releases may be discovered shortly after publication. Cooldowns reduce the chance that Bayview developers become early victims of a compromised version.	Package source control, SCA, package review, or incident response.
Seal Security or equivalent	Curated or patched open source package source that may feed internal repositories.	Packages with difficult remediation paths or where cleanroom/backported packages reduce risk.	Repository management, network blocking, or package manager configuration.
Chainguard Images or equivalent	Hardened image source that may feed an approved internal image repository.	Container base images where Bayview wants controlled, minimal, hardened upstream images.	General developer package repository management.
Socket.dev or equivalent	Package-risk firewall or policy layer before packages reach developers or internal caches.	Higher-risk ecosystems where malicious packages, typosquats, risky install scripts, or suspicious maintainers are a concern.	The internal repository manager itself.

Artifactory vs. Nexus

Artifactory and Nexus solve the same core problem for this roadmap: centralizing package and artifact access so developers do not pull directly from public registries. The choice should be based on ecosystem fit, operating model, existing tooling, support burden, and developer experience.

Decision Area	JFrog Artifactory	Sonatype Nexus Repository
Core overlap	Universal repository manager for internal packages, third-party dependencies, build outputs, containers, and artifacts.	Universal repository manager for internal packages, third-party dependencies, build outputs, containers, and artifacts.
Product center of gravity	Broader artifact, build metadata, release, distribution, and DevOps platform orientation across the JFrog ecosystem.	Repository management aligned with Sonatype open source governance, policy, firewall, and lifecycle tooling.
Package strategy	Strong fit when Bayview wants one enterprise artifact system for many formats, container registries, promotion flows, and API automation.	Strong fit when Bayview wants clear hosted, proxy, and group repository patterns, especially where Sonatype or Maven practices are familiar.
Security integration	Best evaluated with JFrog security features, access controls, audit logs, and any external package-risk firewall tooling.	Best evaluated with Sonatype policy controls, Repository Firewall options, access controls, audit logs, and SCA integrations.
Operations	May be attractive for high-scale artifact storage, rich metadata, build traceability, replication, and distribution patterns.	May be attractive for focused repository administration and teams already aligned to Sonatype governance workflows.
Decision guidance	Prefer if Bayview standardizes on JFrog or wants Artifactory as the main artifact system of record.	Prefer if Bayview standardizes on Sonatype or wants Nexus Repository plus Sonatype policy controls.

Bayview should select one primary enterprise repository manager unless there is a clear operational reason to keep both. Running both can be justified during migration, acquisition integration, or business-unit separation, but it creates duplicated policy, routing, exception, logging, and support work.

Adoption Phases

Phase 0: Identify Languages and Package Managers

Suggested timing: Weeks 0-4

Objectives

- Understand which coding languages and package ecosystems Bayview developers use.
- Identify where developers currently resolve packages directly from public repositories.

Actions

- Use Wiz, source control, CI/CD configuration, developer documentation, endpoint telemetry, and engineering input to inventory languages and package managers.
- Identify use of npm, PyPI, Maven, Gradle, NuGet, Go modules, RubyGems, Cargo, public container registries, operating system package managers, and shell-based installers.

- Identify package manager configuration files such as `.npmrc`, `pip.conf`, `pyproject.toml`, `requirements.txt`, `pom.xml`, `settings.xml`, `build.gradle`, `nuget.config`, `go.mod`, `Gemfile`, `Cargo.toml`, and devcontainer or Dockerfile package installation paths.
- Map applications and repositories to owners and stakeholder groups.
- Prioritize high-use and high-risk ecosystems for migration first.

Exit Criteria

- Initial language and package manager inventory exists.
- Direct external package repository use is known for priority teams or ecosystems.
- Owners are identified for priority repositories and developer workflows.

Phase 1: Select the Internal Repository Pattern

Suggested timing: Weeks 2-6

Objectives

- Decide how Bayview will provide internally managed package repositories.
- Define what counts as an approved package source for each ecosystem.

Actions

- Select the primary enterprise repository manager or repository pattern, such as Artifactory, Nexus, cloud-native registries, or an approved combination.
- Define hosted, proxy, mirror, and grouped repository patterns for priority ecosystems.
- Decide whether cleanroom or hardened sources such as Seal Security or Chainguard should feed internal repositories for selected use cases.
- Decide whether Sonatype Repository Firewall, Socket.dev, or an equivalent package-risk firewall should sit upstream of the repository manager, in developer workflows, or both.
- Define dependency cooldown requirements, including which ecosystems need a cooldown, how long new package versions should be delayed, and how emergency exceptions are approved.
- Define authentication, authorization, audit logging, retention, backup, disaster recovery, and platform ownership requirements.
- Define the minimum package request workflow for packages not yet available internally.

Exit Criteria

- Repository platform decision is documented.
- Priority ecosystems have an approved internal repository pattern.
- Repository firewall and dependency cooldown decisions are documented.
- Platform owner, support model, and exception owner are identified.

Phase 2: Configure and Migrate Developer Workflows

Suggested timing: Weeks 4-12

Objectives

- Make internal package repositories the default developer path.
- Reduce migration friction before broad blocking begins.

Actions

- Publish standard package manager configuration for priority ecosystems.
- Update approved developer environment images, onboarding documentation, and repository templates to use internal repositories.
- Cache or pre-stage commonly used packages needed by priority teams.
- Update source-controlled configuration where appropriate, such as `.npmrc`, `pip.conf`, `settings.xml`, `nuget.config`, and container registry settings.
- Create a support channel for missing packages, broken builds, authentication issues, and migration questions.
- Track teams and repositories that have migrated.

Exit Criteria

- Priority developer workflows can install packages from internal repositories.
- Common package manager setup is documented.
- Missing package requests have a usable intake process.

Phase 3: Block Direct External Package Repository Access

Suggested timing: Weeks 8-16

Objectives

- Move from guidance to enforcement.
- Prevent developers from bypassing internally managed repositories.

Actions

- Create an allowlist of approved internal package repositories and approved exceptions.
- Use endpoint, DNS, proxy, firewall, CASB, or network egress controls to block direct access to public package repositories where practical.
- Alert before blocking for selected pilot teams if needed to reduce disruption.
- Enforce internal repository use in developer workstations, virtual desktops, Codespaces or dev containers, and development-related CI jobs where applicable.
- Review direct-download patterns such as curl-to-shell installers, GitHub release downloads, public container pulls, and package manager fallback behavior.
- Document emergency and vendor-specific exception paths.

Exit Criteria

- Priority developer groups are blocked from direct public package repository access or have approved exceptions.
- External package repository bypass attempts are visible.
- Exceptions are time-bound and assigned to owners.

Phase 4: Operate, Measure, and Expand

Suggested timing: Months 4-8 and ongoing

Objectives

- Keep the control working after migration.
- Expand from priority teams to the broader developer population.

Actions

- Review internal repository logs, block logs, exception lists, and package request volume.
- Expand internal repository requirements to remaining developer groups and ecosystems.
- Tune package availability, caching, authentication, and documentation based on developer feedback.
- Periodically review whether Sonatype Repository Firewall, dependency cooldown tooling, Seal Security, Chainguard, Socket.dev, or equivalent services should be added or tuned for specific upstream risk reduction.
- Update the Secure Software Standard to require developers to use only approved internally managed package repositories.
- Report progress to engineering and security leadership.

Exit Criteria

- Internal repository usage is standard across active developer teams.
- Direct public package repository access is blocked by default for developer package manager traffic.
- Exceptions are reviewed and trending down.
- The Secure Software Standard contains the internal package repository requirement.

Minimum Policy Language

Developers must configure package managers and development environments to resolve packages only from Bayview-approved internally managed package repositories. Direct resolution from external public package repositories is prohibited unless an approved, documented, time-bound exception exists.

Metrics

- Percentage of developer package downloads served by internally managed repositories.
- Number of direct public package repository access attempts blocked or alerted.
- Percentage of priority repositories with approved package manager configuration.
- Number of ecosystems migrated to internal repositories.
- Number of missing package requests opened, approved, denied, and aged.
- Number of package source exceptions by owner, age, and expiration date.
- Number of packages blocked, quarantined, or warned by repository firewall policy.
- Number of newly published package versions delayed by dependency cooldowns.
- Mean time to fulfill approved package availability requests.
- Number of developer teams migrated to internal repository use.

Open Decisions

- Which repository manager or repository pattern should be the primary enterprise standard?
- Which ecosystems should migrate first?
- Which public package repositories and package manager endpoints should be blocked first?
- Which repository firewall or equivalent package-risk policy layer should Bayview use?
- Which ecosystems should receive dependency cooldowns, and what cooldown window should apply?
- Which team owns repository administration and developer support?

- Which team owns network, DNS, endpoint, or proxy enforcement?
- Where will package source exceptions be recorded and reviewed?
- Should Seal Security, Chainguard, Socket.dev, or equivalent services be included in the initial rollout or evaluated later?
- What target date should Bayview set for blocking direct external package repository access?

References

- [SCRATCH Developer Supply Chain Security](#)
- [Supply Chain Security](#)
- [Seal Security](#)
- [Chainguard Images](#)
- [Socket.dev Firewall](#)
- [JFrog Artifactory](#)
- [Sonatype Nexus Repository](#)
- [Sonatype Repository Firewall](#)
- [Dependency Cooldowns](#)